

# L2 - Database Caching Strategies Using Redis

---

**Estudiante:** Carol Araya Conejo **Carnet:** 2024089174

---

## 1. Explique los desafíos que se enfrentan en la construcción de sistemas distribuidos que requieren baja latencia (de acuerdo con la lectura)

- Se tienen consultas con procesamiento lento, esto quiere decir que se pueden utilizar técnicas de optimización, no obstante, existen tiempos de recuperación accediendo al disco más los tiempos en los que se procesa la consulta; debido a ello siempre habrá un límite físico en la velocidad.
  - Costo al escalar -> Cuando se habla de lecturas de datos grandes, pueden ser costosas, y además se pueden ocupar réplicas de la misma lectura en la base de datos, bajando el rendimiento.
  - Simplificación al acceso de datos -> Las bases de datos relacionales no tienen un acceso optimizado a los datos, por lo que hay casos en los que las apps van a tener que acceder a vistas o a alguna estructura para poder recuperar data y así no bajar el rendimiento.
- 

## 2. Explique los tipos de caching, ¿Cuál considera es el más adecuado?

- **Database-integrated caches:** Ofrecen una cache integrada la cual se administra dentro del motor de la base de datos con capacidad de escritura. Y la misma actualiza la cache de forma automática. Una desventaja es estar limitados a la memoria disponible y tiene un uso específico; no puede ser usada para otros fines.
- **Local caches:** Almacena la data que es frecuentada dentro de la aplicación, hace que la recuperación de la misma sea más rápida ya que elimina el tráfico de red que se genera con la recuperación de los mismos datos.
  - Una desventaja es que la información almacenada dentro de un local cache no puede ser compartida con otra, por lo que afecta si se trabaja en un ambiente donde es prioritario compartir información. Y en caso de haber una desconexión, los datos de la memoria cache se pierden.
- **Remote caches:** Es una instancia dedicada a almacenar la data en memoria de la cache, almacenados en servidores dedicados, por lo que se puede acceder a ella de distintos medios y repara las mayorías de las desventajas de una Local Cache. Su solicitud tiene un promedio de latencia menor a realizarla en una disk-based database. Además, la misma es un sistema aparte que las

aplicaciones usan para acceder a consultas; la cache no está conectada directamente.

- Considero que las Remote caches son el tipo de caching más adecuado, ya que actualmente las aplicaciones suelen trabajar con múltiples servidores, lo que hace que este tipo de caché brinde mayor flexibilidad de uso

---

### 3. Explique las diferencias entre los patrones de caching.

- **Cache-aside:**

- Actualiza los datos después de la solicitud.
- Su forma de trabajar es: si los datos están disponibles en la cache, es retornada a la solicitud. De lo contrario, si no lo está, se consulta a la base de datos, se llena la cache con los datos y la misma se retorna a la solicitud.
- Solo contiene los datos que la aplicación solicita, manteniendo un costo bajo.
- Tiene una implementación sencilla y con ganancias de rendimiento.

- **Write-Through:**

- Actualiza los datos de forma inmediata cuando la base de datos principal es actualizada.
- Esta cache está actualizada con la base de datos principal; con ello hay más probabilidad de que los datos solicitados estén en la cache, dando una buena experiencia de usuario.
- Y el rendimiento de la base de datos es óptimo porque se evita realizar lecturas en la base de datos.
- Los datos que no son solicitados con tanta frecuencia también son guardados en la cache, siendo más grande y más costosa.

| Característica                | Cache-aside                                                                                                                                        | Write-Through                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Actualización de datos</b> | Después de la solicitud, solo cuando se requiere                                                                                                   | Inmediata al actualizar la base de datos                               |
| <b>Flujo de trabajo</b>       | Se verifica la caché: <ul style="list-style-type: none"><li>• Si está, se retorna</li><li>• Si no, se consulta la BD y se llena la caché</li></ul> | Cada actualización de la BD se refleja automáticamente en la caché     |
| <b>Contenido de la caché</b>  | Solo los datos solicitados por la aplicación                                                                                                       | Datos solicitados y también datos menos usados; la caché es más grande |
| <b>Costo / tamaño</b>         | Bajo, porque almacena solo datos necesarios                                                                                                        | Más alto, porque almacena muchos datos                                 |

|                               |                                                           |                                                                      |
|-------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------|
| <b>Rendimiento</b>            | Ganancia de rendimiento moderada; implementación sencilla | Rendimiento de lectura muy bueno; base de datos menos cargada        |
| <b>Experiencia de usuario</b> | Puede haber cache miss la primera vez                     | Alta probabilidad de que los datos estén en caché, mejor experiencia |

**4. Explique que es cache eviction, ¿Por qué es relevante para el rendimiento de un sistema?**

- La cache eviction se genera cuando la cache excede el límite de memoria configurada o está sobrellenada; esto hace que el motor escoja keys para desalojar y así administrar la memoria, basado en políticas seleccionadas.
- Es relevante a su rendimiento ya que evita que la cache se sobrecargue, además algunas políticas permiten tener los datos que son más útiles en la memoria y así se evita estar accediendo a la base de datos, que relentiza el proceso, así como reduce la presión en la base de datos ya que va a distribuir eficientemente la carga. Y gracias a la eviction se puede llegar a saber si es necesario escalar la cache.